Taller 02: Señales analógica - uso del ADC y PWM

1. Objetivos

- Configurar y utilizar las entradas del conversor analógico-digital (**ADC**) del microcontrolador RP2040.
- Realizar lecturas de magnitudes físicas internas (temperatura del chip) y eléctricas (voltaje de alimentación V_{SYS} y un potenciómetro externo).
- Implementar salidas digitales con modulación por ancho de pulso (**PWM**) para un control proporcional de potencia (brillo de un LED).
- Desarrollar el algoritmo en **MicroPython** utilizando bucles de lectura, conversión de unidades y escalado de señales.

2. Materiales y Equipamiento

- 1x Raspberry Pi Pico (con firmware MicroPython instalado).
- 1x Potenciómetro de $10\ \text{k}\Omega$ (o similar).
- 1x LED de cualquier color.
- 1x Resistencia de 220Ω a 470Ω (para protección del LED).
- Protoboard y cables de conexión (Jumpers Dupont).
- Entorno de desarrollo (IDE Thonny) en PC.

3. Marco Teórico Breve

- **ADC del RP2040:** Cuenta con un conversor de 12 bits (valores de 0 a 4095), pero en MicroPython las lecturas se escalan automáticamente a **16 bits** (0 a 65535).
  - `ADC(0)` (GP26): Entrada analógica general.
  - `ADC(1)` (GP27): Entrada analógica general.
  - `ADC(2)` (GP28): Entrada analógica general.
  - `ADC(3)` (GP29): Conectado internamente a un divisor resistivo para medir V_{SYS}.
  - `ADC(4)` : Conectado al sensor de temperatura interno del chip.
- **Fórmula del Sensor de Temperatura Interno:**

$$\text{Temperatura}\ (^\circ\text{C}) = 27 - \frac{V_{ADC} - 0.706}{0.001721}$$

  Donde $V_{ADC} = \text{Lectura_16bits} \times \left(\frac{3.3}{65535}\right)$
- **PWM (Pulse Width Modulation):** Permite simular una señal analógica variando el ciclo de trabajo (*duty cycle*). En MicroPython para RP2040, el rango de *duty cycle* se maneja de **0 a 65535** (0% a 100%).

4. Esquema de Conexiones (Hardware)

Esquema preliminar orientativo

```
             RASPBERRY PI PICO
           +------------------+
           |                  |
 3.3V OUT -| [Pin 36]         |- GP26 (ADC0) <--- Cursor Potenciómetro
      GND -| [Pin 38]         |- GP15 (PWM)  ---> [R 220Ω] ---> [LED (+)] --
-> GND
  Vsys IN -| [Pin 39] (ADC3)  |
           +------------------+

POTENCIÓMETRO:
  Extremo 1 -> 3.3V (Pin 36)
  Cursor    -> GP26 (Pin 31)
  Extremo 2 -> GND (Pin 38)
```

5. Desarrollo del Código en MicroPython

Escriba y guarde el siguiente código en la Raspberry Pi Pico:

```
import machine
import time

# --- Configuración de Entradas Analógicas (ADC) ---
adc_pot = machine.ADC(26)       # GP26 -> Potenciómetro
adc_vsys = machine.ADC(29)      # GP29 (ADC3) -> Tensión Vsys
adc_temp = machine.ADC(4)       # ADC4 -> Sensor de Temperatura Interno

# --- Configuración de Salida Proporcional (PWM) ---
led_pwm = machine.PWM(machine.Pin(15))
led_pwm.freq(1000)              # Frecuencia establecida en 1 kHz

FACTOR_CONV = 3.3 / 65535       # Factor de conversión a Voltios
# puede utilizar la función print para ir mostrando por terminal los valores
adquiridos

#
```

6. Guía de Actividades y Cuestionario del Alumno

1. **Prueba de Funcionamiento Base:**
   - Compile y ejecute el programa.
   - Gire el potenciómetro y verifique que el brillo del LED varía linealmente en función de la posición del cursor.
   - Modifique la posición del potenciómetro y observe cómo cambian en la consola REPL los valores medidos en voltios y el valor asignado al PWM.

2. **Verificación de Mediciones (V_{SYS} y Temperatura):**
    - Compare el valor de V_{SYS} leído en la consola con el valor medido usando un multímetro directo en el Pin 39 de la tarjeta. ¿A qué se debe el factor de multiplicación $\times 3$ en la línea de código?
    - Coloque suavemente el dedo sobre el chip RP2040 (sin provocar estática). ¿Se observa un incremento progresivo en la lectura del sensor de temperatura?
3. **Ejercicios de Modificación del Código (A resolver por el alumno):**
    - **Ejercicio A (Inversión del Control):** Modifique el código para que el LED funcione de manera *inversamente proporcional*: a mayor voltaje en el potenciómetro, menor brillo en el LED.
    - **Ejercicio B (Control por Umbral de Temperatura):** Agregue una condición lógica en la cual, si la temperatura del chip supera los $35\ ^{\circ}\mathrm{C}$, el LED comience a parpadear como advertencia, desactivando el control por potenciómetro.
    - **Ejercicio C (Frecuencia PWM):** Varíe la frecuencia del PWM a $10\ \mathrm{Hz}$ (`led_pwm.freq(10)`). ¿Qué sucede visualmente con la luz del LED cuando modifica el potenciómetro? Explique la diferencia entre cambiar la frecuencia y el ciclo de trabajo (*duty cycle*).

7. Informe de Laboratorio Requerido

El informe final presentado por el alumno debe contener:

- Diagrama de conexiones en Fritzing, Tinkercad, Kicad o CAD del tipo EDA.
- Capturas de pantalla de la consola REPL con las lecturas en tiempo real.
- Código final de los **Ejercicios de Modificación** debidamente comentado.
- Conclusiones sobre la precisión del ADC de 12/16 bits y la aplicación práctica del control PWM.